Cognito as Auth Server

Title	**Cognito as Auth Server**
Identifier	TRA-CASM-0003 Cognito as Auth Server
Security Domain	CASM
Status	Approved
Version	**v1.0**
Technology landscape	AWS Cognito, API Gateways
Parent	N/A
SubArticles	N/A

Contents

Introduction

This Technical Reference Architecture (TRA) defines the standard for using AWS Cognito as an Authorization Server (AuthZ server) in federated authentication scenarios with external Identity Providers (IdPs).

In the context of OAuth 2.0 and OpenID Connect (OIDC), the Authorization Server is the component responsible for issuing security tokens (Access Tokens, Refresh Tokens, ID Tokens) to client applications on behalf of a resource owner (the user).

This document focuses exclusively on this Authz Server capability, the mechanisms of token issuance, validation, and the required application integration patterns. It deliberately excludes the broader topics of identity lifecycle management, governance, and provisioning, focusing purely on the OAuth 2.0/OIDC protocol implementation.

This architecture will co-exist with other approved Authorization Servers, and this TRA provides the definitive technical guidance for development teams choosing to leverage Cognito for this purpose.

The primary goals of this architecture are:

- **Standardize API Security:** Provide a single, consistent, and secure method for user-facing applications to gain access to protected internal APIs.
- **Developer Enablement:** Offer clear, prescriptive patterns for developers to integrate new and existing applications (SPAs, web apps, mobile, services) with a central Authz Server.

← Contents

Scope

The main scope of this architecture is defined to cover 100% AWS environment scenarios.

In-Scope

- **100% AWS Applications consuming Business APIs:** This TRA is focused on 100% AWS applications that consume Business APIs.
- **OAuth 2.0 Application Integration Patterns:** Prescribed patterns for modern application architectures, including server-side web applications, single-page

applications (SPAs), mobile and native desktop applications, and backend services such as microservices, daemons, and batch processes.

- **Authorization Server Role:** This TRA is focused on the role of Cognito as an Authorization Server (issuing access tokens for APIs).

Out-of-Scope

- **Identity Lifecycle:** User provisioning, de-provisioning (JML), and attribute management.
- **Identity Governance (IGA):** Access reviews, entitlement management, and SoD policies.
- **Legacy Protocols:** SAML, WS-Federation, or Kerberos. The focus is strictly on modern, token-based protocols (OAuth/OIDC).
- **Cognito Infrastructure:** The underlying deployment, global distribution, or maintenance of the Cognito service (which is managed by AWS).

Use cases

For the definition of this reference architecture we have considered the following application architecture use cases:

- **SPA → Business Apis:** Client Applications consumes different Business APIs (i.e. A bank office application that consumes Accounts and Cards APIs). The app authenticates a corporate user and calls a Business API (Resource Server) on behalf of that user.
- **SPA→API→Backend:** Each Client Application only consumes its own APIS. (i.e. an Administration console). The SPA calls the APIs of its own business domain.
- **Service → Service (Client Credentials):** Client Application is a backend service that consumes other Service API's on its own name.

Constraints

All solutions designed under this TRA must operate within the following constraints:

- **Protocol:** All authorization flows must use the OAuth 2.0 protocol.
- **Grant Types:** Auth Code and Client Credentials are the only grant types covered by this TRA.
- **Identity Source:** This architecture assumes identities are already present and managed within the IdP.
- **Platform Dependency:** This architecture is dependent on the AWS Cognito cloud service. Its performance, reliability, and feature set are governed by AWS's service delivery. In the same way, all implementation details are relative to this platform.
- **Libraries:** While not mandatory, the Security Context Manager (SCM) are strongly recommended for client-side integration to ensure correct protocol implementation and token cache management.

← Contents

Quick Decision Guide for Architects

This section serves as a rapid assessment guide to determine if Cognito is a suitable Authorization Server for your use case.

- For information on DPoP see: DPoP-(Demonstration-of-Proof-of-Possession).aspx

Core Concepts and Components

To clearly understand the proposed technical architecture, we must introduce some key concepts related with Cognito platform:

Authorization Server vs Identity Provider

While a single platform like AWS Cognito can fulfill both roles, it is architecturally critical to distinguish between its function as an Authorization Server and its function as an Identity Provider. This separation of concerns is a foundational principle for building clean, secure, and maintainable application architectures.

- **Identity Provider (IdP):** The IdP is responsible for authentication. Its primary role is to manage identities, verify credentials, and assert that a user is who

they claim to be. When using OIDC, the IdP's primary output is an ID Token, which proves an authentication event occurred.

- **Authorization Server (Auth Server):** The Auth Server is responsible for delegated authorization. Its role is to issue access tokens to client applications after the user (Resource Owner) has been authenticated and has granted consent. The access token is the key that unlocks a protected resource (API). The API should only be concerned with the validity and permissions within the access token, not how the user was authenticated.

This TRA is exclusively focused in the configuration and use of AWS Cognito in its capacity as an Authorization Server.

Cognito application Permissions Model

Amazon Cognito is used as the OAuth 2.0 Authorization Server for the platform. It provides mechanisms to control access to protected API resources through **OAuth scopes** and to manage user roles through **Cognito User Pool groups**.

These two mechanisms address different authorization concerns and are applied at different layers of the architecture

Within an **Application Object**, Cognito supports two complementary permission models that can be leveraged depending on the authorization use case:

1. **Delegated Permissions (OAuth Scopes)**
2. **User Groups (Role-Based Authorization)**

Delegated Permissions (OAuth Scopes)

Delegated permissions follow the standard OAuth 2.0 scope-based authorization model. They define what actions a client application is allowed to perform when acting **on behalf of an authenticated end user**.

This model is primarily used in **user-centric flows**, such as the **Authorization Code flow**.

Usage in the Architecture

- Scopes are defined at the **resource server level** and represent API-level permissions (e.g., Mail.Read, Mail.Write).
- During the OAuth flow, the client application requests a set of scopes.
- Cognito evaluates the request and issues an **Access Token** containing the approved scopes.

Authorization Semantics

Delegated permissions are typically **contextualized to the authenticated user**. This means that even if a scope allows access to a given resource type, the actual access is limited to resources owned by or associated with the current user.

Example:

If an application is granted the Mail.Read scope, it can only read the emails of the user who completed the authorization flow, not emails belonging to other users.

Token Representation

When delegated permissions are granted, the **Access Token** includes:

- An scope (scp) claim, containing the list of OAuth scopes granted to the client application.
- A cognito:groups claim, containing the groups (roles) assigned to the authenticated user.

Summary

- **OAuth scopes** are used to model delegated, user-centric permissions and control access to API resources.
- **Cognito groups** are used to represent business roles and profiles and support role-based authorization.
- Both mechanisms are complementary and may coexist within the same Access Token.
- Final authorization decisions are enforced by downstream services based on token claims and contextual rules.

Summary

Aspect	OAuth Scopes (Delegated Permissions)	Cognito Groups (Role-Based Authorization)
Primary purpose	Define what actions a **client application** is allowed to perform **on behalf of an authenticated user**.	Represent **business roles or user profiles** assigned to users.
Standard / Specification	Part of the OAuth 2.0 specification.	Cognito-specific feature (not part of OAuth 2.0).

Aspect	OAuth Scopes (Delegated Permissions)	Cognito Groups (Role-Based Authorization)
Definition location	Defined on the resource server and requested by the client application.	Defined and managed in the Cognito User Pool.
Level of granularity	Action- and resource-oriented (e.g., `Mail.Read`, `Files.Write`).	Role- or profile-oriented (e.g., `Admin`, `Finance`, `Support`).
Binding	Bound at **authorization time** through the OAuth consent process.	Bound to the user identity and managed administratively.
Token representation	Included in the Access Token as the `scope` (scp) claim.	Included in the Access Token as the `cognito:groups` claim.
Authorization focus	Controls **what the application can do** for the user.	Controls **what the user is allowed to do** from a business perspective.
Evaluation point	Evaluated by the resource server on each API request.	Evaluated by downstream services or business logic.
Typical use cases	User-centric APIs, per-user resource access, delegated access scenarios.	Administrative permissions, privileged operations, business rule enforcement.
Relationship to OAuth	Core authorization mechanism in OAuth flows.	Orthogonal to OAuth; complements scopes but does not replace them.
Architectural scope	Part of the reference OAuth authorization architecture.	Managed at the component level; outside the core OAuth architecture.

Consent Management

Overview

When Amazon Cognito operates as an OAuth 2.0 / OpenID Connect Authorization Server, consent management is handled in an implicit and centrally administered manner. Cognito does not provide native capabilities for explicit end-user consent, interactive approval, or user-driven consent revocation.

Instead, consent is established through administrative pre-authorization of application clients and their associated scopes.

This model is well aligned with controlled corporate environments, but it does not support scenarios that require explicit user consent or third-party access delegation.

1. Consent Model

Amazon Cognito does not implement user-facing consent screens or interactive approval workflows.

- Consent is considered **implicit**.
- Authorization is **pre-approved at the platform configuration level**.
- End users are not involved in approving or denying access requested by applications.

2. Scope of Consent

Consent in Cognito is managed entirely through administrative configuration:

- **Application Client level**
  - Definition of allowed OAuth flows
  - Definition of permitted scopes for each client
- **Resource Server level**
  - Definition of available OAuth scopes

The end user has no control over:

- Approving requested scopes
- Rejecting scopes
- Limiting scope usage at runtime

3. Token Issuance Behavior

During OAuth authorization flows:

1. The user is authenticated by Cognito.
2. Cognito evaluates the requested scopes against the client configuration.
3. Cognito issues tokens containing all requested scopes that are authorized for the application client.

No runtime consent validation or user confirmation step is performed during token issuance.

4. Revocation and Control Mechanisms

Amazon Cognito does **not support**:

- User-level consent revocation
- Consent lifecycle management
- Consent audit trails

Control over access is enforced exclusively through administrative and technical mechanisms, including:

- Enabling or disabling application clients
- Modifying allowed scopes for a client
- Updating resource server scope definitions
- Access token and refresh token revocation

5. Architectural Implications

Cognito's consent model assumes:

- **Trusted application clients**
- **Controlled execution environments**

As a result, Cognito is **not suitable as an authorization server** for:

- Open ecosystems
- Third-party application integrations
- Use cases with regulatory or legal requirements for **explicit user consent**

6. Extensibility Considerations

Any requirement for explicit consent must be:

- Implemented **outside of Cognito**
- Handled at higher architectural layers (application, API gateway, or middleware)

Amazon Cognito does **not provide native extensibility** mechanisms for consent management or user-driven authorization decisions.

Summary

Amazon Cognito provides a static, administratively managed consent model that is well suited for closed, corporate architectures.

However, meeting advanced requirements such as explicit consent, delegated third-party access, or regulatory compliance requires additional controls beyond the identity provider.

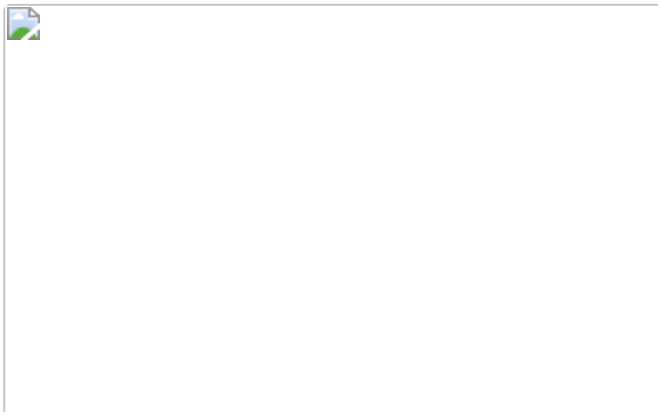
← Contents

Architectural Overview

At its core, this architecture uses Cognito as the central "trust broker" between an user, the application they are using, and the API that application needs to call.

Key Actors

- **Resource Owner (User):** The user who owns the data and grants permission.
- **Client Application:** The application the employee is using. It requests access on the user's behalf.
- **Authorization Server (Cognito):** The central engine. It performs two critical functions:
  - **Authenticates the users** (via OIDC).
  - **Issues tokens** (Access Token, Refresh Token) to the Client Application after successful authentication and consent, in accordance with policy (Conditional Access).
- **API gateway:** The gateway that protects the API. It trusts Cognito, receives the Access Token from the Client Application and validates it to grant access.
- **Resource Server (API):** The protected API. It receives the token with the required claims to apply business logic and execute fine grain authorization validations.



← Contents

Quick Integration Guide for Architects

The integration of applications with Cognito acting as an OAuth Server must be performed through the proper configuration of Cognito in conjunction with the application settings, the configuration of the Identity Provider (IdP) via an OIDC integration, and the definition of the claims to be propagated. This may require the development of a Lambda function to collect and enrich the data needed for inclusion in the access token.

Application configuration.

For the configuration of the application that needs to consume APIs exposed under Cognito's security umbrella, the following parameterization is required:

- **AccessTokenValidity**: The access token time limit. After this limit expires, your user can't use their access token
- **AllowedOAuthFlows**: The OAuth grant types that you want your app client to generate for clients in managed login authentication: code, client_credentials.
- **AllowedOAuthFlowsUserPoolClient**: True
- **AllowedOAuthScopes**: The OAuth, OpenID Connect (OIDC), and custom scopes that you want to permit your app client to authorize access with.
- **AnalyticsConfiguration**: The user pool analytics configuration for collecting metrics and sending them to your Amazon Pinpoint campaign.
- **AuthSessionValidity**: Amazon Cognito creates a session token for each API request in an authentication flow. No mandatory
- **CallbackURLs**: A list of allowed redirect, or callback, URLs for managed login authentication.
- **ClientName**: A friendly name for the app client that you want to create.
- **DefaultRedirectURI**: The default redirect URI.
- **EnablePropagateAdditionalUserContextData**: When true, your application can include additional UserContextData in authentication requests.
- **EnableTokenRevocation**: Activates or deactivates token revocation in the target app client.
- **ExplicitAuthFlows**: The authentication flows that you want your user pool client to support. No required.
- **GenerateSecret**: When true, generates a client secret for the app client.
- **IdTokenValidity**: The ID token time limit.
- **LogoutURLs**: A list of allowed logout URLs for managed login authentication.
- **PreventUserExistenceErrors**: When ENABLED, suppresses messages that might indicate a valid user exists when someone attempts sign-in.
- **ReadAttributes**: The list of user attributes that you want your app client to have read access to.
- **RefreshTokenRotation**: The configuration of your app client for refresh token rotation.
- **RefreshTokenValidity**: The refresh token time limit.
- **SupportedIdentityProviders**: A list of provider names for the identity providers (IdPs) that are supported on this client.
- **TokenValidityUnits**: The units that validity times are represented in

- **UserPoolId**: The ID of the user pool where you want to create an app client.
- **WriteAttributes**: The list of user attributes that you want your app client to have write access to.

More information in: https://docs.aws.amazon.com/cognito-user-identity-pools/latest/APIReference/API_CreateUserPoolCl

Amazon Cognito User Pool Configuration

When Amazon Cognito is used as an **OAuth 2.0 and OpenID Connect Authorization Server**, the **User Pool** acts as the identity directory, authentication authority, and token issuer.

Proper configuration of the User Pool is a foundational step to support OAuth flows, scope-based authorization, and secure token issuance.

When an Amazon Cognito User Pool is configured to **federate with an external OpenID Connect (OIDC) Identity Provider**, Cognito acts as a federation broker.

In this architecture:

- The **external IdP** is responsible for authenticating the user.
- Amazon Cognito remains the **OAuth 2.0 / OIDC Authorization Server** for client applications and APIs.
- Cognito issues its own tokens and enforces authorization based on its configuration.

This approach enables integration with enterprise identity systems while maintaining a consistent OAuth interface for consuming applications.

Role Separation

Component	Responsability
External OIDC IdP	User authentication, primary identity validation
Amazon Cognito User Pool	Identity brokering, user mapping, token issuance
Client Applications	OAuth flows, token consumption

Component	Responsability
Resource Servers	Token validation and authorization enforcement

Cognito does not pass through tokens issued by the external IdP.
Instead, it validates the external IdP response and issues **Cognito-native tokens**.

User Directory and Schema in a Federated Setup

User Representation

When federation is enabled:

- Each authenticated external user is represented as a **User Pool user**.
- The user is uniquely identified by:
  - External IdP identifier (e.g., sub)
  - IdP name (provider identifier)

Schema and Attribute Mapping

Cognito maps claims received from the external IdP into User Pool attributes.

- Standard attributes (e.g., email, name) can be mapped directly.
- Custom attributes can be used to store:
  - External user IDs
  - Tenant or organization identifiers
  - Business metadata

Design considerations:

- Use immutable external identifiers to prevent identity collisions.
- Avoid duplicating sensitive attributes unless strictly necessary.
- Normalize attributes across different IdPs if multiple providers are supported.

User Pool Creation and Configuration

User Pool Setup

1. Navigate to **AWS Console → Amazon Cognito → User Pools.**
2. Create or update a User Pool (e.g., Production-Users).
3. Enable federation and select **OpenID Connect** as the external provider type.

Environment isolation remains a best practice (separate pools per environment).

External OIDC Identity Provider Configuration

OIDC Provider Settings

When registering the external IdP in Cognito, the following parameters must be configured:

- **Issuer URL** (e.g., https://login.example.com)
- **Authorization endpoint**
- **Token endpoint**
- **UserInfo endpoint** (optional)
- **Client ID and Client Secret** issued by the external IdP
- **Scopes** requested from the external IdP (e.g., openid, profile, email)

Cognito uses these endpoints to:

- Redirect users for authentication
- Validate ID tokens
- Retrieve user claims

Cognito Domain Configuration

Purpose in a Federated Flow

The Cognito domain remains the single OAuth entry point for client applications, even when authentication is delegated to an external IdP.

- Client applications redirect users to the Cognito authorization endpoint.
- Cognito redirects users to the external IdP as part of the authentication step.
- After successful authentication, Cognito completes the OAuth flow and issues tokens.

Domain Configuration

- Configure a Cognito-hosted domain prefix or a custom domain.
- The domain is embedded in:
  - Redirect URIs
  - Token issuer (iss) claim
  - Client and resource server trust configuration

Token Content

Access Tokens may include:

- scp – OAuth scopes granted to the client
- cognito:groups – User roles
- User attributes mapped from the external IdP

Security Considerations

- Trust the external IdP issuer and signing keys.
- Validate token lifetimes and session management behavior.
- Protect against attribute spoofing by controlling claim mappings.
- Use short-lived access tokens to limit exposure.

Runtime Architecture

This section details the runtime flows and security mechanisms required for integration.

Key Endpoints

All integrations will use next endpoints, which are OIDC compliant. The authoritative source for tenant-specific endpoints is the OIDC discovery document:

- **Discovery Endpoint:**

```
https://<cognito-domain>.auth.
<region>.amazoncognito.com/.well-known/openid-configuration
```

- **Authorization Endpoint:**

  ```
  https://<cognito-domain>.auth.
  <region>.amazoncognito.com/oauth2/authorize
  ```

- **Token Endpoint:**

  ```
  https://<cognito-domain>.auth.
  <region>.amazoncognito.com/oauth2/token
  ```

- **JWKS Endpoint:**

  Found in the discovery document (e.g., jwks_uri). Contains the public keys to validate token signatures.

  https://<cognito-domain>.auth.<region>.amazoncognito.com/.well-known/jwks.json

Access Token Definition

The formal definition of Cognito access tokens, aligned with the AWS access token claims reference, with the correspondent RFCs as well as with the requirements imposed by the Common Application Security Model can be found.

Authorization Flows

Authorization Code Flow with PKCE.

As we said, for use cases where an SPA acts as a public client and consumes shared Business APIs on behalf of a user, this architecture mandates the use of **Authorization Code Flow with Proof Key for Code Exchange (PKCE)**. This extension to OAuth 2.0 mitigates the risk of authorization code interception and removes the need for the insecure Implicit Grant flow.

In a standard Auth Code flow, a *client_secret* is used to exchange the code for a token. Since an SPA cannot hold a secret, PKCE substitutes this with a dynamic, ephemeral secret generated at runtime.

SPA Interaction Modes: Redirect vs. Pop-up vs. Silent

If implementing this flow using the **SCM Library**, architects must choose the appropriate interaction mode. This decision impacts User Experience (UX) and reliability.

OIDC Scopes

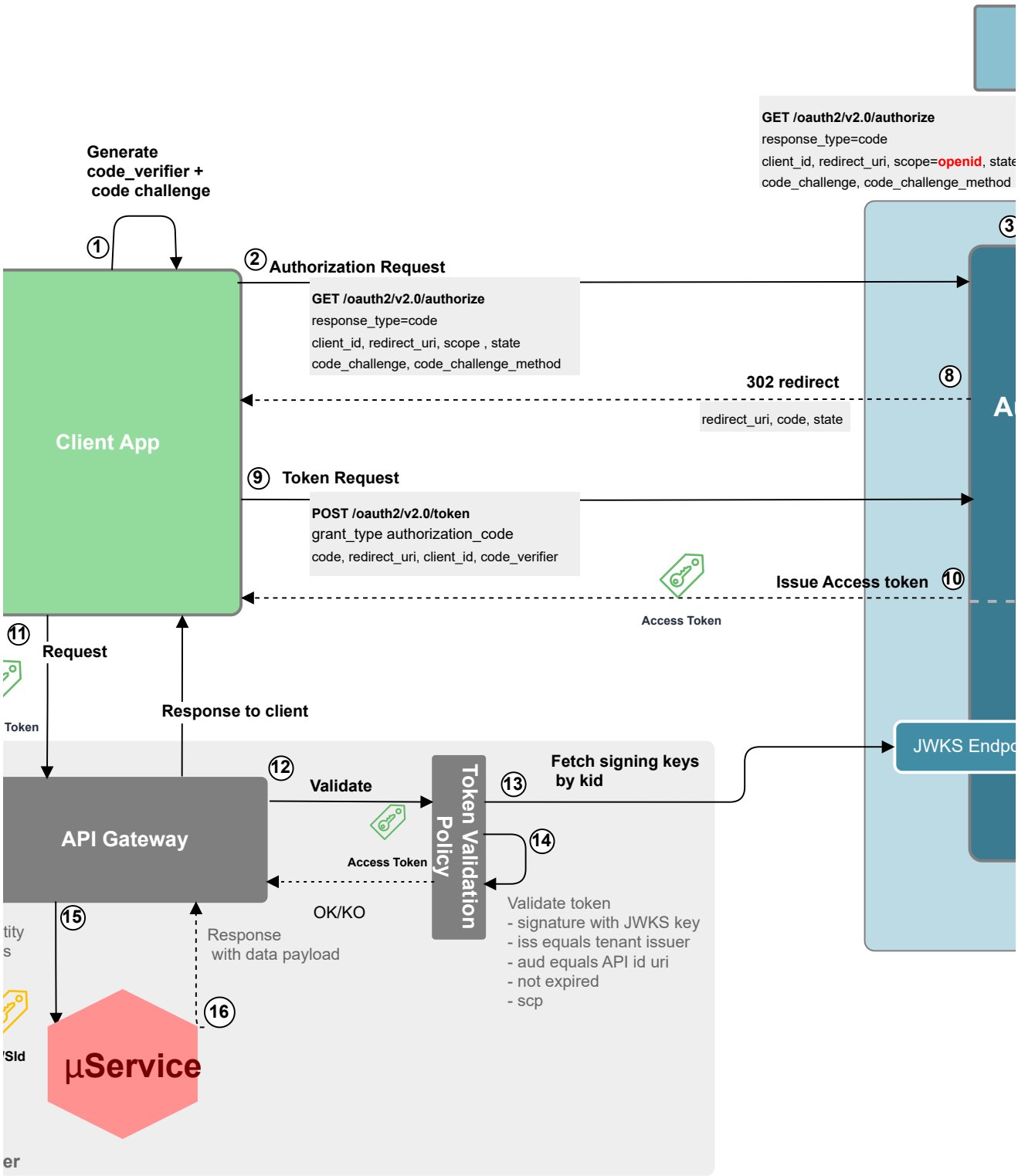
For a standard implementation of Authorization code Grant on Cognito, you **must** include at least a special scope:

- **offline_access (Required for Refresh Tokens):** By default, Cognito Access Tokens live for about 60 minutes. When they expire, your user would be forced to log in again. This scope explicitly requests a **Refresh Token**. `offline_access` is supported **Authorization Code flow.** The `offline_access` scope **does not appear** in the `scp` claim of the access token. Its purpose is **only to trigger refresh token issuance**. Cognito will **not** issue a refresh token unless: the App Client has *"Allow Refresh Token Auth Flow"* enabled and a refresh token expiration is configured.

Refresh Token lifetime is fixed per App Client

- Refresh token TTL is configured at the App Client level
- It cannot be shortened or extended dynamically per user or per request
- Changing the TTL only affects newly issued refresh tokens

Auth Code Flow + PKCE (Redirect)



Prerequisites:

- The Public client is registered in Entra ID with its application registration.
- The target API is registered with an Application ID URI and exposes delegated permissions (Scopes).

The flow can be summarized as follows:

1. **Code Challenge Generation:** The client creates a high-entropy random string (*code_verifier*) and hashes it (SHA-256) to produce a *code_challenge*.

2. **Authorization Request:** The Client application sends a request to Cognito's /authorize endpoint to obtain permission to access the Resource Server, including the code_challenge.

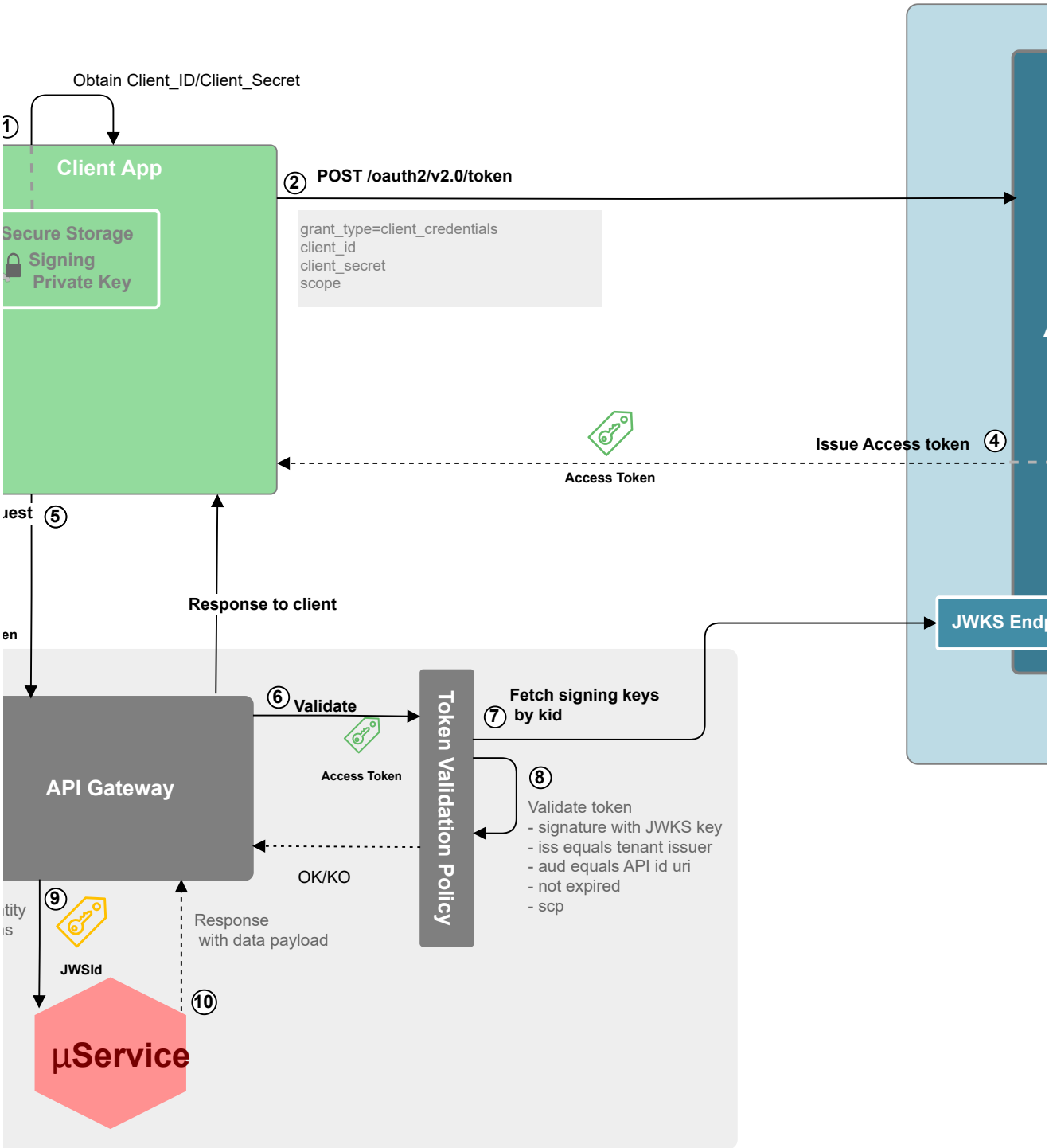
   GET /oauth2/authorize?
response_type=code&client_id=...&code_challenge=
[hash]&code_challenge_method=S256

   The client application requests an Access Token for each of the APIs it needs to consume, with the specific required scopes. The scopes in the Authorization request will have the form ***api://<APP Id>/<scope>*** **.** Each authorization request permits the Client App to request more than one scope at the same time (blank separated).

3. Cognito interacts with the IdP for managing Single Sign On process via OIDC. Cognito calls to IDP through the authorize endpoint.
4. **Authentication :** If the employee has not an SSO session on Entra Id or MFA is required, the authentication process will be launched. This step is considered out of scope for this TRA, but it establishes the user session necessary for the authorization flow.
5. The IdP authenticates to the user and return a code to Cognito.
6. Cognito calls to the IdP token endpoint with the code.
7. IdP Returns the IDToken with the user information.
8. After authentication, Cognito returns an Auth Code to the client.
9. **Token Request:** After authentication, Cognito returns an Auth Code. The client exchanges this code for a token, sending the original code_verifier in the request body.
10. POST /oauth2/token... code=[code]&code_verifier=[original_string]
11. **Token Issuance:** Cognito hashes the received code_verifier. If the result matches the stored code_challenge from step 2, it proves the client exchanging the code is the same one that initiated the request. If policies are satisfied, upon successful authorization, Cognito issues an access token (and refresh token) to the Client.
12. **API Call:** The Client application calls the Resource Server (API), presenting the access token in the Authorization: Bearer <token> HTTP header.
13. API Gateway invokes Token Validation Policy.
14. Token Validation Policy gets the Cognito Public key from its JWKs endpoint, and caches it.
15. Token Validation Policy validates the access token's signature (using Cognito signing public key), issuer, audience, and claims

16. API receives the token with Scopes and roles claims and validates authorization as applicable.
17. If all validations are passed, the request is served.

Client Credentials flow



Prerequisites:

- The confidential client is registered in Cognito with its application registration.

The flow can be summarized as follows:

1. **Client_ID/Client_Secret:** The client obtain the cliend_ID and Client_Secret to prove its identity.
2. **Authorization Request:**
    - The client sends a POST to the Cognito token endpoint: POST /oauth2/token
    - It sets grant type to client_credentials.
    - Includes the client_secret.
3. **Authorization server validates the request**
    - Validate client_ID and Client_Secret.
    - Validate client application status and permissions:
        - The client app is enabled and allowed to use client credentials
        - The requested scopes or resource are permitted
4. **Token Issuance:** If policies are satisfied, upon successful authorization, Cognito issues an access token (and refresh token) to the Client.
5. **API Call:** The Client application calls the Resource Server (API), presenting the access token in the Authorization: Bearer <token> HTTP header.

   Token Validation & Access:
6. API Gateway invokes Token Validation Policy.
7. Token Validation Policy gest Cognito signing Public key from its JWKs endpoint, and caches it.
8. Token Validation Policy validates the access token's signature (using Cognito signing public key), issuer, audience, and claims
9. API receives the token with Scopes and roles claims and validates authorization as applicable.
10. If all validations are passed, the request is served.

Access Token Validation

When a client request reaches the Resource Server, the Cognito issued access token must be validated, typically via an API Gateway policy.

This Policy will verify among others:

- Verify timestamp claims
- Verify Origin & Recipient
- Signature Validation
- Authorization Claims (Scopes, ...)

← Contents

Deployment and Scalability

- **Cloud-Native Service:** AWS Cognito is a globally distributed, multi-tenant cloud service managed by AWS. It is designed for high scalability and handles billions of authentications daily. The underlying infrastructure scales automatically to meet demand, requiring no capacity planning from the organization.
- **Global Distribution:** Cognito operates out of geographically distributed datacenters. This ensures low latency for authorization requests by routing traffic to the nearest datacenter and provides geo-redundancy.
- **Programmatic Configuration (DevOps):** All aspects of application registration, permission configuration, and policy management can be automated via the AWS Cognito API. This enables integration into CI/CD pipelines, allowing for consistent, repeatable, and auditable deployments of application identity configurations.

← Contents

Performance and Reliability

- **Service Level Agreement (SLA):** Microsoft guarantees at least 99.9% availability for AWS Cognito. The SLA measurement is based on the ability of users to successfully authenticate and for Cognito to issue tokens. The worldwide SLA performance is publicly reported by Microsoft.
- **High-Availability Architecture: Amazon Cognito is a regional service**: each *User Pool* stores its data in the region where it is created; there is no automatic global replication. Within that region, **Cognito is designed to be**

resilient and to leverage multiple *Availability Zones* (AZs) to provide fault tolerance and high availability. There is an important distinction between the **data plane** (authentication, token issuance and validation) and the **control plane** (administrative operations such as creating or updating pools and configuration); understanding this separation is key when planning recovery strategies and managing dependencies.

- **Failover and Redundancy:** Amazon Cognito provides **automatic redundancy and failover within a single AWS region**. It tolerates **instance- and Availability Zone–level failures** without customer intervention. It does **not provide native multi-region failover**. Full regional resilience **depends on the overall system architecture**, not on Cognito alone.
- **Tenant-Level Monitoring:** Tenant-level monitoring in Amazon Cognito is primarily based on **per–User Pool CloudWatch metrics and logs**. Metrics provide visibility into **health and traffic volume**, while logs explain the **root cause of authentication issues**. **Lambda triggers and external dependencies** (SES, SNS, federated IdPs) are essential components of effective tenant-level observability. In mission-critical environments, effective monitoring is **intentionally designed**, not simply enabled by default..

← Contents

Anexes

OIDC Federation between AWS Cognito (Service Provider) and Microsoft Entra ID (Identity Provider)

1. Objectives

To configure OIDC Federation between AWS Cognito (Service Provider) and Microsoft Entra ID (Identity Provider) for Single Page Applications (SPAs).

Ensure custom attributes (like UPN) originating from Azure are successfully propagated through Cognito and injected into the OAuth 2.0 Access Token consumed by backend APIs.

2. Summary

- **Architecture:** SPA -> Cognito -> Entra ID.
- **Protocol:** OIDC (OpenID Connect).
- **Critical Requirement:** The backend API needs some specific user and security claims (UPN,client_id, etc...) inside the **Access Token**.
- **Key findings:**
  - Cognito OIDC Client requires explicit “profile” scope to fetch attributes from Entra ID /userinfo endpoint.
  - App Clients need explicit "Read" permissions on Cognito for custom attributes.
  - Access Token enrichment will be provided by Lambda Trigger V2 as indicated below.
  - The Scope format is <API_Id>/<Scope> (e.g: Accounts/Read)
  - Claim “sub” contains the Cognito User Pool’s internal user uuid.

3. Step-by-Step Configuration Guide

STEP 1: Microsoft Entra ID Configuration (Identity Provider)

Register Cognito as a trusted application and expose necessary claims.

1. **Register the Application:**
   - Go to **Entra admin center** > **Identity** > **Applications** > **App registrations** > **New registration**.
   - Name: Cognito-Integration.
   - Supported account types: **Single tenant**.
   - Redirect URI: Leave blank for now (we will add the Cognito Domain later).
   - Click **Register**.

2. **Generate Secrets:**
    - Navigate to **Certificates & secrets** > **Client secrets** > **New client secret**.
    - Copy the OIDC Client Secret Value immediately.

3. **Token Configuration (Ensure Data Availability):**
    - Navigate to **Token configuration** > **Add optional claim**.
    - Token type: **ID**.
    - Select, for example: upn, email, family_name, given_name.
    - Add them. (Accept the "Turn on Microsoft Graph permissions" prompt).

4. **API Permissions:**
    - Navigate to **API permissions**.
    - Ensure User.Read (Microsoft Graph) is present.
    - Click **Grant admin consent for [Your Organization]**.

STEP 2: AWS Cognito User Pool Setup

Create the directory and the schema to hold the external data.

1. **Create/Edit User Pool:**
    - Go to **Amazon Cognito** > **User pools**.
    - Create a pool (e.g., Production-Users).
    - **Cognito Domain:** Configure a domain prefix (e.g., my-org-auth) or leave it as default.

2. **Define Custom Attributes (The Container):**
    - Navigate to the **Sign-up experience** tab.
    - Section **Custom attributes**.
    - Click **Add custom attributes** as needed.
    - Example:
        - **Name:** “entraid_upn” (Cognito will save it as “custom:entraid_upn”).
        - **Type:** String.
        - **Mutable: Yes** (Check this! If it is immutable, mapping will fail).
        - Max length: 256.

STEP 3: OIDC Provider Configuration.

Connect Entra ID and authorize the data transfer.

1. Navigate to **Authentication** >**Social and External Providers**.

2. Click **Add identity provider** > **OpenID Connect (OIDC).**
3. **Provider name:** EntraID.
4. **Client ID:** Paste the *Application (client) ID* from Entra ID.
5. **Client secret:** Paste the *Secret Value* from Step 1.
6. **Authorized scopes:**
    - **CRITICAL STEP:** You must explicitly list the scopes “openid email profile” here.
    - Without profile, Entra ID filters out several attributes from the UserInfo response.
7. **Attribute request method:** GET.
8. **Issuer URL:** https://login.microsoftonline.com/<YOUR_TENANT_ID>/v2.0
    - *Note:* Using /v2.0 ensures standard OIDC compliance.

STEP 4: Attribute Mapping

Map Azure claims to Cognito attributes.

1. Navigate to **Attribute mapping** (inside Identity Provider configuration).
2. Configure the mapping table as needed. For example:

User Pool Attribute	OpenID Connect Attribute	Note
email	email	Standard.
custom:entraid_upn	upn	Or preferred_username if upn is missing.
given_name	given_name	
family_name	family_name	Ensure profile scope is active.

STEP 5: API Scopes Registration.

Declare API scopes.

1. Navigate to **Branding** > **Domain** > **Resource Servers** > Create resource server

2. **Resource server name:** Enter a friendly name for the resource server.
3. **Resource server identifier:** Enter a unique identifier for the resource server. **Requests for resource server scopes must include the identifier and the scope name.**
4. **Custom scope->Add Custom Scope**: Define the required scopes.

STEP 6: App Client Configuration.

Allow the SPA to receive the data.

1. Navigate to **Applications**-> **App clients** -> Create **App Client**
2. Select **Application type (Web, SPA, Mobile, Machine to Machine**), and set Name and Return **URLs** (e.g., http://localhost:3000), and click Create App Client.

3. Navigate to **Login Pages:**
    - **Identity Providers:** Select EntraID.
    - **OAuth 2.0 grant types:** Authorization code grant.
    - **OpenID Connect scopes:** openid, email, profile for OIDC, and the Business API scopes required (previously created as resource server).
    - Save changes.

4. **Navigate to Attribute permissions:**
    - Scroll down to **Attribute read and write permissions**.

- Click **Edit**.
- Find previously created custom claims (i.e. “custom:entraid_upn”).
- **Check "Read"**: Required for the Lambda and the Token to see the data.
- **Check "Write"**: Required for the IdP to save the data.
- Save changes.

Step 7: Lambda Trigger V2 (Access Token Enrichment)

Inject the Custom Claims into the Access Token for the backend API.

1. **Go to Authentication > Extensions > Lambda Triggers > Add Lambda Triggers**

2. **Select:**
   - **Trigger type:** Authentication (Control sign-in, customize tokens, and log analytics events.)
   - **Authentication:** Pre token generation trigger (Modify claims in ID and access tokens.)
   - **Trigger event version:** Basic features + access token customization for user and machine identities.

3. **Create Lambda Function> Create Function:**
    - Author from scratch
    - Function Name: AccessTokenClaimAdapter
    - Runtime: Node.js 24.x.
    - Create Function

Go to **Code** tab (ES Module syntax):

Paste the following JavaScript snippet, and click **Deploy** button.

```
export const handler = async (event) => {
    // Log para debug (verás esto en CloudWatch)
    console.log("Evento recibido:", JSON.stringify(event, null, 2));

    const newClaims = {};

    // Validación defensiva: Solo agrega si el atributo existe en el request
```

```
    if (event.request.userAttributes) {
        if (event.request.userAttributes['custom:entraid_upn']) {
            newClaims['upn'] = event.request.userAttributes['custom:entraid_upn'];
            newClaims['oid'] = event.request.userAttributes['custom:entraid_oid'];
        }
        if (event.request.userAttributes['email']) {
            newClaims['email'] = event.request.userAttributes['email'];
        }
    }
    newClaims['static_claim'] = "static value";
    // Construcción de la respuesta V2 estricta
    event.response = {
        claimsAndScopeOverrideDetails: {
            accessTokenGeneration: {
                claimsToAddOrOverride: newClaims
            }
        }
    };
    // Retornar el evento completo
    return event;
};
```

Go back to Extensions formulary (point 2)

- On Lambda Function, select your recently created Lambda function, and click **Add Lambda trigger**

Step 8: Finalize Entra ID Redirect URI

1. Go back to **Entra ID** > **App registrations** > **Authentication**.
2. Add **Web** Platform.
3. **Redirect URI:** https://<YOUR_COGNITO_DOMAIN>.auth.<REGION>.amazoncognito.com/oauth2/idpresponse
4. Save.

4. Test integration:

- We have user Postman for simulating the OAuth Authorizwsation Code Flow:

It helps to build and launch the Authorization request to the Auth Server:

https://eu-north-11vyurzdzi.auth.eu-north-1.amazoncognito.com/oauth2/authorize?
response_type=code&
client_id=1h5ohkmaeh4gekoqlqjfm51ogb&
scope=Accounts%2FRead&
redirect_uri=https%3A%2F%2Foauth.pstmn.io%2Fv1%2Fcallback&
code_challenge=hwFDYi192hc34-h6-uuY0qgAU7kZwFVa12qW5jzC7iE&
code_challenge_method=S256

- Cognito redirects us to Entra ID for authentication.

- Once Authentiocated, the control is returned to Cognito, and the client will proceed to call cognito /Token endpoint, returning the Access Token.
- We can see the customized claims present on the returned Access Token.

Scope Management in Amazon Cognito (when acting as an OAuth 2.0 Authorization Server)

Below is a complete, structured treatment of **scope management** for Amazon Cognito used as an OAuth 2.0 / OpenID Connect authorization server — suitable for architecture docs, implementation guides, or an ADR. It covers concepts, configuration, token behavior, enforcement, lifecycle, mapping to roles/groups, operational concerns, examples, and recommendations.

1. Conceptual overview

- **Scope (OAuth 2.0)**: a granular string-based permission that describes an action or resource the client app may perform or access on behalf of a user

(e.g., `mail.read`, `files.write`).

- **Purpose in Cognito**: scopes are the primary mechanism for **delegated authorization** — they tell resource servers *what* the client is allowed to do for the authenticated user.
- **Cognito behavior**: Cognito evaluates client configuration and requested scopes during OAuth flows and includes granted scopes in the issued **Access Token** under the `scp` claim.

2. Where scopes live in Cognito

1. **Resource Server(s)** (recommended pattern)
   - You create one or more *Resource Servers* within a User Pool.
   - Each Resource Server defines a **resource identifier** (audience) and a set of **scopes** (name + description).
   - Example: Resource Server `api://mail` defines scopes `mail.read`, `mail.send`.
2. **App Client configuration**
   - Each App Client (application) can be configured with allowed OAuth flows and which scopes it is permitted to request.
   - Cognito will only grant scopes that are both requested in the authorization call and allowed by the App Client configuration.
3. **In tokens**
   - The granted scopes are embedded in the Access Token claim `scp` (space-delimited string).
   - `offline_access` may be requested to obtain refresh tokens, but `offline_access` is not present in `scp`.

3. How to define scopes (console / IaC)

Console (high level)

1. Open **Amazon Cognito → User Pools → [your pool] → App integration → Resource servers**.
2. Create a Resource Server:
   - `Identifier` (e.g., `api://my-service`)
   - `Name`, `Description`
3. Add scopes for that resource:
   - `Scope name`: `mail.read`
   - `Scope description`: `Read user mail messages`

Infrastructure as Code

- Use CloudFormation / Terraform resources for `AWS::Cognito::UserPoolResourceServer` (CloudFormation) or `aws_cognito_resource_server` (Terraform) to declare resource server and scopes in source-controlled config.

4. Requesting scopes (client flow)

- When a client performs the authorization request, it includes a `scope` parameter listing requested scopes:

  ```
  GET /oauth2/authorize?
  response_type=code&client_id=...&redirect_uri=...&scope=openid
  profile api://my-service/mail.read mail.send
  ```
- Cognito grants scopes that:
  - exist in the resource server definitions,
  - are permitted for the App Client, and
  - are allowed by any platform policy.
- Result: Access Token includes `scp` claim with granted scopes:

  ```
  "scp": "openid profile api://my-service/mail.read"
  ```

5. Token content & claims

- **Access Token**
  - `scp`: space-separated list of granted OAuth scopes (delegated permissions).
  - `aud`: audience (client or resource server identifier).
  - `iss`, `exp`, `sub`, etc.
  - `cognito:groups` may also appear if user is in groups.
- **ID Token**
  - Intended for client/user identity; may include OIDC claims (profile, email).
  - Not used for resource access checks.

Important: Resource servers **must** validate the Access Token (signature, `iss`, `aud`, `exp`) and then inspect `scp` to make authorization decisions.

6. Enforcement patterns at the Resource Server

1. **JWT validation**
   - Validate signature with JWKS from Cognito (`/.well-known/jwks.json`).
   - Validate `iss` (issuer matches your user pool domain).
   - Validate `aud` (contains expected client or resource id).
   - Validate `exp` (not expired).

2. **Scope check**
    - Define required scope(s) per endpoint (e.g., `mail.read` required for `GET /messages`).
    - Check that the `scp` claim contains the required scope. Reject with 403 if missing.
3. **Owner / context check**
    - For user-centric resources, combine `scp` with ownership info: verify `sub` equals resource owner unless an admin scope/role permits cross-user access.
4. **Role augmentation**
    - Optionally use `cognito:groups` or custom claims (added via pre-token Lambda) to apply role-based rules in addition to scope checks.
5. **Least privilege**
    - Enforce minimal scope per route; don't grant broad scopes unless needed.

7. Scope lifecycle & management

- **Creation & versioning**: Scopes are created at resource server configuration time. Use IaC for reproducibility.
- **Granting**: Occurs at runtime when the client requests scopes and Cognito determines they are permitted.
- **Revocation / change**:
    - Changing scope definitions or App Client allowed scopes only affects tokens issued *after* the change.
    - Existing tokens still carry prior `scp` values until expiration.
- **Auditing**:
    - Log authorization requests, granted scopes, client_id, and `sub` for traceability.
    - Use CloudWatch / CloudTrail for admin-level config changes.

8. Consent model and scopes

- Cognito uses an **administrative (implicit) consent model**:
    - Consent is configured by administrators via App Client permissions.
    - There is **no built-in interactive user consent screen** for approving individual scopes during authorization (unless you build custom UI). This matters when scopes imply third-party delegation.
- Implication: Scopes should be carefully controlled at the App Client level; do not rely on end-user revocation in Cognito.

9. Best practices & security recommendations

- **Namespace scopes** by resource to avoid collisions (e.g., `api://orders/read`).
- **Define fine-grained scopes** rather than permissive, coarse scopes.
- **Map scopes to API endpoints** explicitly and enforce checks in middleware.
- **Combine scopes with group/role claims** for admin operations (e.g., require `admin` group + `users.read` scope).
- **Short-lived access tokens** and conservative refresh token TTLs to reduce risk when tokens are leaked.
- **Use pre-token Lambda** to add non-sensitive custom claims (namespaced) required by the resource server.
- **Avoid putting sensitive PII in tokens**; include identifiers and let backend fetch details.
- **Automate scope configuration** via IaC and review scope changes in PRs.
- **Audit and log** granted scopes and authorization flows.

10. Common pitfalls & caveats

- **Assuming `offline_access` appears in `scp`** — it doesn't. It only signals the issuance of a refresh token.
- **Relying on existing tokens after scope removal** — tokens preserve old scopes until they expire.
- **Large `scp` payloads** — too many scopes or very long claims can bloat JWTs; keep tokens compact.
- **Not namespacing custom claims** — this can collide with standard claims or third-party consumers.
- **Not validating audience correctly** — tokens may be issued for different resource servers; always validate `aud`.

11. Example: minimal authorization middleware (pseudo-code)

```
function requireScope(accessToken, requiredScope) {  claims = validateJwt(accessToken)  // signature, iss, aud, exp  scopes = (claims.scp || "").split(" ")  if (!scopes.includes(requiredScope)) {    throw new Forbidden("Missing scope: " + requiredScope)  }  // optional owner check:  // if (isUserResource && claims.sub !== resourceOwnerId) { ... } }
```

12. Operational checklist for production readiness

- Define resource servers and scopes in IaC
- Configure App Clients with allowed scopes & flows
- Implement token validation middleware in all resource servers
- Apply least-privilege design for scopes
- Configure CloudWatch/CloudTrail monitoring for User Pool changes and auth events
- Implement pre-token Lambda if you need additional claims or normalization
- Decide refresh token TTL strategy (desktop vs mobile vs public clients)
- Document scope-to-endpoint mappings in API docs

13. When to extend or replace Cognito scope model

Consider extra layers if you need:

- **User-driven consent screens** and consent revocation per scope (Cognito lacks native support).
- **Refresh token rotation** and reuse detection (Cognito does not support rotation natively).
- **Advanced delegation models** (third-party consent flows or delegated access marketplaces).

In those cases, you can:

- Add a **consent service** in front of Cognito to manage explicit consent and store user approvals.
- Implement a **BFF/token broker** that issues application-managed tokens with rotation semantics while using Cognito for authentication.

Client Application Management

Client Applications will be managed on Entra Id too as *APP Registration* Objects. They can be managed using the Entra Id UI or programmatically (Graph API) providing the following params:

Functional Area	Parameter / Setting (Portal Label)	Microsoft Graph Property	Recommended Value /
Identity	**Display name**	*displayName*	Clear human readable na Contoso APP Client - Pro

Functional Area	Parameter / Setting (Portal Label)	Microsoft Graph Property	Recommended Value /
	Application (client) ID	*appId*	Read-only. System-gener
	Object ID	*id*	Read-only. System-gener
Authentication	**Web redirect URIs**	*web.redirectUris*	i.e. "https://web.contoso.cor oidc"
	SPA redirect URIs	*spa.redirectUris*	i.e. "https://app.contoso.(
	Front-channel logout URL	*web.logoutUrl*	i.e. "https://web.contoso.cor oidc",
Credentials	**Client secret**	*passwordCredentials*	
	Certificate / public key	*keyCredentials*	
Outbound Permissions	**API permissions (delegated scopes)**	*requiredResourceAccess[]. resourceAccess (type=Scope)*	See below examples.

Functional Area	Parameter / Setting (Portal Label)	Microsoft Graph Property	Recommended Value /
	API permissions (application roles)	*requiredResourceAccess[]. resourceAccess (type=Role)*	See below examples.
Token Claims Config	**Optional claims (ID token)**	*optionalClaims*	i.e. "email"
	Group claims emission	*groupMembershipClaims (on service principal)*	
Ownership / Governance	**Owners**	*/owners* relationship	Assign app team membe automation identity

Grant Type Management

Entra ID does not provide explicit grant type configuration at the client application level. Instead, grant types are implicitly enabled or disabled based on various application configuration settings.

Grant / Flow	How It Becomes Enabled (Portal / Graph)	Client Type Required	Additional Prerequisites
Authorization Code + PKCE	Any valid redirect URI in Web (*web.redirectUris*), SPA (*spa.redirectUris*), or Mobile/Desktop; no special toggle, PKCE parameters supplied at runtime.	PKCE is mandatory enforced for Public Clients; Optional for Confidentials	Delegated permissions; admin consent as needed

Grant / Flow	How It Becomes Enabled (Portal / Graph)	Client Type Required	Additional Prerequisites
Client Credentials	Presence of secret (*passwordCredentials*), certificate (*keyCredentials*), or federated identity (*federatedIdentityCredentials*)	Confidential only	Application permissions (appRoles) on target resource API with admin consent.
Refresh Token	Request *offline_access* scope in an eligible flow (Auth Code, Device Code, ROPC).	Public or Confidential	Admin consent for scopes; policy must allow.
JWT Client Assertion (cert-based auth)	Upload certificate (*keyCredentials*)	Confidential	Delegated or Application permissions on target resource API with admin consent.

- Confidential client= app has a secret, certificate, or workload identity federation.
- Public client = no credentials.
- PKCE enforcement is implicit for public client scenarios. However, there is currently no setting in Entra ID to make PKCE “mandatory” (i.e., reject authorization code requests that do not contain a code_challenge) for a confidential application.

Access Token Claims Configuration

Claims within access tokens encapsulate the authorization context, defining identity, permissions, and access conditions. Their correct configuration is fundamental to application security and efficiency. This section examines Entra ID's claim configuration capabilities, implementation scenarios, and technical constraints. As we said, on this TRA we will focus on v2.0 tokens.

Claim Types

Entra ID uses a hierarchy to determine which claims are included in a token. Understanding this is key:

- **Core/Basic Claims:** A minimal set of claims included in every access token (*aud, iss, oid, sub, tid, exp,* etc.). These are largely immutable and form the basis of token validation.
- **Optional Claims:** A pre-defined set of claims you can request to be added to the token (e.g., *ipaddr, upn*).
- **Group Claims:** Configuration to embed a user's security group membership.
- **Claims from User Attributes:** For advanced scenarios, you can create a claim that sources its value from almost any attribute on the user object in Entra ID. This includes:
  - Standard attributes like *employeeId*, *department*, or *jobTitle*.
  - On-premises synced attributes.
  - Custom Directory Extension attributes.

Customization Mechanisms

Entra ID provides three distinct mechanisms to add claims to an access token, each serving a different technical purpose.

- **Optional Claims:** This is the standard mechanism for "opting in" to receive predefined claims that are not included by default (to keep tokens small), or for sourcing claims directly from directory extension attributes. This is configured via the Token configuration blade in the portal or the optionalClaims property in the Application Manifest.
- **Claims Mapping Policies:** This is an advanced, programmatic-first mechanism for "custom claims" sourced from data within Entra ID. It is used to add claims from attributes not available in the standard optional list (e.g., employeeId), to transform claim values, or to omit basic claims. This mechanism overrides any Optional Claims configuration.
- **Custom Claims Providers:** This is an event-driven mechanism for "augmenting" tokens with claims from external systems. It allows Entra ID to make a real-time API call to an external service (e.g., a database) during the token issuance process to fetch and embed external data as claims.

Customization Mechanisms & Use Cases

Optional Claims:

A simple, UI-driven way in the App Registration's "Token configuration" blade to add standard, but not default, claims.

- **When to use :**
  - You need, for example, the user's ***UPN (upn)*** for display or as a unique key (though ***oid*** is preferred for immutability).
  - You need context attributes like the client ***IP address (ipaddr)***.
  - You need to support legacy on-premises applications and require a user's on-premises attribute.
- **Constraints:**
  - The list of available claims is fixed. You cannot create custom claims this way.
  - Adding too many can contribute to token size, but the impact is generally low.

Group Claims

A configuration in the "Token configuration" blade to embed the user's security group memberships (Object IDs) into a groups claim.

- **When to use:**
  - When migrating a legacy application that already uses AD groups for authorization and cannot be easily refactored.
  - For very simple RBAC scenarios where managing App Roles is considered overhead.
- **Constraints:**
  - **Token Bloat:** This is the single biggest risk. If a user is a member of many groups, the token size can exceed limits for HTTP headers, causing application failures.
  - **Overage Claim (*_claim_names*, *_claim_sources*):** To prevent bloat, if a user is in too many groups (over 150 for SAML, 200 for JWT), Entra ID will not embed the groups. Instead, it inserts a special "overage" claim that points to a Microsoft Graph endpoint. The API is then responsible for an extra round-trip to call that endpoint to fetch the user's group list. This adds complexity and latency.
  - **Transitive Membership:** Be aware of which groups you select (Security groups, Directory roles, All groups). Selecting "All groups" can include nested/transitive memberships, dramatically increasing the claim size.
  - **Recommendation:** Avoid group claims for new applications. Use App Roles instead.

Claims Mapping Policies

A JSON policy object, created via PowerShell or MS Graph, that is applied to an application's Service Principal. It allows for advanced transformations.

This method has not been broadly tested, and a technical reference for its use is not yet available.

- **When to use : USE WITH CAUTION. This is a last resort.**
  - **Legacy App Migration:** An existing application expects a claim with a specific name that doesn't match the Entra ID claim name (e.g., it expects ***employee_id*** but the data is in ***employeeId*** on the user object).
  - **Combining Claims:** You need to create a custom claim by joining two user attributes.
  - **Custom Attributes:** You need to emit extension attributes as a claim in the token.
- **Constraints:**
  - **High Complexity:** Difficult to create, manage, and debug. There is no UI support (PowerShell/Graph API only)
  - **Overrides Other Configurations:** A claims mapping policy can override default and optional claims, leading to unexpected token contents if not managed carefully.
  - **Security Risk:** Because you can source claims from almost any attribute on the user object, you must ensure you are not leaking sensitive data (e.g., HR data) into tokens for applications that shouldn't see it.
  - **Use Immutable Sources:** When mapping a claim for authorization decisions, the source attribute MUST be immutable and centrally managed (e.g., employeeId). Using a mutable attribute that a user could potentially influence (e.g., otherMails) is a significant security risk.
  - There exist a set of restricted claims that cannot be modified our used as a source on claim mapping policies.

Mechanism	Configuration Interface	Data Source	Supports Transformation?	Primary Use Cas
Optional Claims (Standard)	App Registration -> Token configuration (UI)	Predefined claims (e.g., ipaddr), Directory Schema Extensions (e.g., extension_..._skypeID)	No (except Group claim formatting)	Adding simple, p claims, or moder extensions.

Mechanism	Configuration Interface	Data Source	Supports Transformation?	Primary Use Cas
	App Registration -> Manifest (optionalClaims)			
Claims Mapping (via Enterprise App UI)	Enterprise Application -> Single Sign-on -> Attributes & Claims (UI)	User Object (e.g., user.extensionAttribute1)	Limited (basic mapping)	The standard pat onPremisesExten (extensionAttribu other simple use
Claims Mapping Policy (Programmatic)	PowerShell / Microsoft Graph (Policy Object creation and assignment)	Any User/Company attribute in Entra ID (e.g., employeeId)	**Yes** (Full transformation via "Join", "Extract", etc.)	Complex, policy- transformations; non-standard cla *within* Entra ID; o claims.

Automation for App Registrations

In Entra ID, the entire configuration of an application registration, including its permissions and properties, is defined in its JSON manifest. This allows its configuration to be treated as code, enabling version control, peer review, and automated deployment.

- **Microsoft Graph API:** The Microsoft Graph API is the authoritative RESTful endpoint for all programmatic management of Microsoft Entra ID resources. All automation tooling for app registrations ultimately interacts with this API. Direct API calls can be used for bespoke automation scripts.
- **Microsoft Graph PowerShell SDK:** This is the officially supported PowerShell module for managing Entra ID resources. It provides a comprehensive set of cmdlets for creating, updating, and deleting application registrations, service principals, permissions, and credentials. It is the recommended tool for imperative scripting tasks.

← Contents

References

- Amazon Cognito user pools
- Amazon Cognito identity pools
- https://docs.aws.amazon.com/cognito-user-identity-pools/latest/APIReference/Welcome.html
- https://docs.aws.amazon.com/cognitoidentity/latest/APIReference/Welcome.html

← Contents